EMBER: Post-Quantum CDN Access Tokens via Epoch-Merkle Batched Root Signatures

Dhavan Arjampudi

***Abstract*—CDNs gate protected objects with signed URLs and cookies, using either an HMAC key shared with every edge or RSA, ECDSA or Ed25519, which a quantum computer would break. Replacing them with per-token ML-DSA or SLH-DSA signatures yields tokens of $3{,}350$ to $22{,}907$ bytes that exceed URL budgets and inflate every segment request. We present EMBER, a CDN profile of Merkle batch signatures: the issuer places all tokens of an epoch in a salted, domain-separated Merkle tree, signs only the root, and each token carries an authentication path. We contribute a threat model stating that HMAC is already post-quantum and that the gain is removing minting secrets from edges; root-dissemination modes for an edge fleet, including root stapling; a security reduction sketch; a measurement study against 13 baselines, among them multi-MAC, delegated Ed25519, KEM-wrapped MAC keys, MTL mode and cached ML-DSA; and a fleet simulator validated against a Go prototype. With 2^{16} tokens per epoch an EMBER token is 503 B, against $3{,}350$ B for ML-DSA-44. Warm verification takes 2.34 µs natively, 43× less than ML-DSA-44 verification but 7.9× HMAC, and one issuer core produces 1.28 million tokens/s. Over HTTP/2 a session needs 0.144× the header bytes of split-cookie ML-DSA-44. Stapling lowers the share of simulated requests that wait for a root to 0.049%. We also report negative results. An adaptive epoch controller performed no better than a fixed 10 ms epoch. Period roll-up removed only 1.4–2.2% of root misses. A hybrid Ed25519+ML-DSA root raised simulated edge CPU per request by 41.6%.**

***Index Terms*—Access tokens, content delivery networks, Merkle trees, ML-DSA, post-quantum cryptography, signed URLs, SLH-DSA.**

I. Introduction

CONTENT delivery networks (CDNs) protect paywalled video, software downloads and other private objects with *access tokens*. An issuer, usually the content owner's authentication service, gives an authorized client a short-lived token that binds a URL prefix, an expiry time and optional client attributes. The client attaches the token to every request as a query parameter (a *signed URL*), a cookie or a header. Any edge server can then check the token locally before serving from cache, without contacting the origin. A single streaming session issues one manifest request and then hundreds of segment requests, so the token is verified hundreds of times per session, at every point of presence (PoP) the client reaches.

Today's token schemes fall into two families. Symmetric schemes, such as Akamai's EdgeAuth tokens [1], append an HMAC [2] under a key that every edge holds. Public-key schemes sign the token: Amazon CloudFront key groups accept RSA-2048 or ECDSA P-256 keys [3], and Google Media CDN uses Ed25519 [4]. HMAC-SHA256 is not threatened by quantum computers, but the minting key sits on every edge. The public-key schemes avoid that, but Shor's algorithm [5] recovers their signing keys from public keys that stay configured in CDN dashboards for years. Authentication has no harvest-now-decrypt-later exposure: a token that expires today cannot be usefully forged later. The motivation for migrating now is the lead time to change token formats, SDKs, edge verifiers and size assumptions before a cryptographically relevant quantum computer (CRQC) exists. Industry roadmaps target post-quantum authentication by about 2029 [6].

The obvious migration, one FIPS 204 or FIPS 205 signature per token [7], [8] encoded as in RFC 9964 [9], does not fit the setting. With a 60-character path, a measured ML-DSA-44 token is $3{,}350$ B, an ML-DSA-65 token $4{,}535$ B and an SLH-DSA-128s token $10{,}598$ B (Section VII-A). These exceed the 2 KB URL length that is common practice, ML-DSA-65 tokens exceed the 4096-B per-cookie minimum of RFC 6265 [10], and every segment request carries the token again. RFC 9964 itself cautions that ML-DSA may be a poor choice where small signatures are required [9].

The standard remedy for large post-quantum signatures is to sign the root of a Merkle tree [11] once and give each message an authentication path. Batch signatures [12], Merkle Tree Ladder (MTL) mode [13], [14], Merkle Tree Certificates [15] and TLS batch signing [16] all follow this pattern. We do not claim the primitive. We study what happens when it is applied to CDN access tokens. In that setting thousands of edges must obtain the signed root before they can verify, tokens must fit URL and cookie budgets, issuers must answer interactive logins quickly, and strong alternatives exist that a fair evaluation must include. Examples are per-CDN multi-MAC tags, short-lived delegated Ed25519 keys, ML-KEM-wrapped per-epoch MAC keys and ML-DSA tokens with an edge verification cache.

Contributions. We present EMBER (Epoch-Merkle Batched, Edge-verifiable, Root-signed tokens) and make the following contributions.

1) **Threat model and requirements** (Section III). We state plainly that HMAC tokens are already post-quantum secure. The case for asymmetric tokens is trust distribution: a compromised edge should not be able to mint tokens for other edges, other CDNs or other tenants. We bound that benefit honestly, because a compromised edge can always serve what it caches.
2) **A CDN token profile of Merkle batch signatures** (Section IV). The profile specifies a 30-byte binary

D. Arjampudi is with Andhra University, Visakhapatnam, India (e-mail: *[editable]*).

[Editable: funding statement, ORCID, corresponding-author note.]

header plus path, issuer-salted leaves, tweakable-hash domain separation, epoch-bounded validity, per-token revocation, tenant pooling and a hybrid root option. It comes with a security theorem whose proof sketch reduces unforgeability to the root signature and to a tweakable-hash property, and an analysis of 16-byte node truncation.

3) **Root dissemination for an edge fleet** (Section V). We define and analyze pull-on-miss with coalescing, push, two-timescale period roll-up and *root stapling*, in which the client sends the signed root once, on its first request to an edge.
4) **A measurement study against 13 baselines** (Sections VI and VII). The study uses native OpenSSL 3.5.4 microbenchmarks, real HPACK-encoded bytes per session, and a Go edge/issuer prototype measured in separate network namespaces over an emulated link.
5) **A validated fleet simulator and its negative results** (Section VII). The simulator matches the prototype's root misses per token within 4.4% in all 24 tested configurations. We report where it fails. We also report the hypotheses the data refuted. The most important is that our adaptive epoch controller did not beat a fixed epoch, so we demote it to a design guideline under a rule fixed before the experiment.

All code, raw data and figure scripts are released (Data Availability statement).

II. Background and Related Work

A. CDN Access Tokens

A CDN token consists of *claims* and an *authenticator*. Typical claims are a URL prefix, an expiry time and optionally a client IP range. Media CDN serializes claims as colon- or ampersand-separated fields followed by `Signature=`, an unpadded base64url Ed25519 signature [4]. EdgeAuth uses `exp=...~acl=...~hmac=...` [1]. Tokens travel in three places. In a query string, an 8000-octet minimum is recommended for URL support [17], and 2 KB is common legacy practice. In a cookie, RFC 6265 requires at least 4096 B per cookie [10]. In a header, servers commonly cap total header size, a configuration assumption we state where we use it. In HTTP/2, HPACK [18] can index a repeated cookie, but only if the cookie fits in the dynamic table next to the other changing headers. Section VII-A shows that this decides whether large tokens are affordable.

Macaroons [19] add attenuation to HMAC bearer tokens but keep HMAC's shared-secret trust. DPoP [20] binds tokens to a client key and is orthogonal to how the token itself is authenticated. Post-quantum Privacy Pass [21] provides unlinkable tokens, but at 85–175 KB per token, for a different goal (anonymity).

B. Post-Quantum Signatures

FIPS 204 standardizes ML-DSA, with signatures of 2420, 3309 and 4627 B for ML-DSA-44/65/87, and FIPS 205 standardizes SLH-DSA, whose 128s and 128f parameter sets have 7856-B and 17,088-B signatures [7], [8]. RFC 9964 registers ML-DSA for JOSE and COSE [9]. Measurements of post-quantum TLS [22]–[25] show that signature *size*, more than CPU, drives the cost of post-quantum authentication. CDNs already deploy ML-KEM key exchange at the edge and toward origins [26], [27]. KEMTLS replaces handshake signatures with KEMs [28], [29], but tokens are verified offline and have no handshake to which that idea could apply. Hybrid signatures combine a classical and a post-quantum scheme. We use the AND-combiner, in which both signatures must verify, analyzed by Bindel et al. [30].

TABLE I
Merkle batching approaches compared with EMBER.

	Root signature per message	Verifiers	Size constraint
Batch sig. [12]	carried in full	one TLS client	none
MTL mode [14]	ladder fetched separately	DNS resolvers	DNS message
MTC [15]	tree head distributed ahead	browsers	TLS flight
EMBER	disseminated by the CDN: pull, push, roll-up, **stapled**	thousands of edges, ∼150 requests per token	URL (2 KB practice), cookie (4096 B)

C. Merkle Batch Signing

Merkle's tree authentication [11] underlies every scheme in this subsection. Aguilar-Melchor et al. [12] formalize batch signatures, in which one inner signature covers a Merkle tree of messages, and prove unforgeability and privacy. In their model each batch signature carries the root signature and is evaluated for TLS. TLS batch signing [16] follows the same pattern. MTL mode [13], [14] signs *ladders* over a growing message series and authenticates messages by a *condensed* signature, a path to a ladder rung. It targets settings, such as DNSSEC [31], in which verifiers check a subset of a large, evolving series. Merkle Tree Certificates (MTC) [15], [32] batch certificate issuance under signed tree heads that relying parties obtain ahead of time. MTAC [33] profiles the same structure for AI-agent identity, and its authors state that it specifies no runtime authorization. Certificate Transparency [34] defines the tree shape that MTC and our implementation follow. SPHINCS$^+$/SLH-DSA introduced the tweakable-hash framework with address-based domain separation [35] that our node hashing reuses.

D. What is New Here

Searches of the literature and of IETF drafts up to November 2026 found no published scheme that applies Merkle batching to CDN signed URLs or cookies. Table I lists the differences in the setting. Our contribution is therefore not the primitive. It is (i) a token profile that meets URL and cookie budgets, with epoch-bounded validity and revocation, (ii) dissemination of signed roots through the CDN itself,

including stapling, and (iii) a measurement study that compares the result with strong symmetric, delegated and cached alternatives. Broader surveys of post-quantum migration [36], [37] list such application-layer artifacts as open problems.

III. System and Threat Model

A. Entities

Issuer I. The tenant's authentication service, or an issuer operated by the CDN, holds the signing key sk. sk is an ML-DSA-44/65, SLH-DSA-128s/f or hybrid Ed25519 + ML-DSA-44 key, identified by a key id kid. *Root service.* An origin object path from which signed epoch roots and period bundles are served. It is itself cacheable through the CDN's shield tier. *Edges* $E_1, \ldots, E_E$ hold pk, a root cache and a revocation set. *Clients* authenticate to I (for example at a paywall), obtain a token and then issue a manifest request and K segment requests, normally sticky to one edge.

B. Why Not Simply HMAC?

HMAC-SHA256 tokens are secure against quantum adversaries. Their weakness is that every verifier can also mint. (i) A single edge compromise (a memory disclosure, an insider, a misconfiguration, partner hardware) yields a key that mints tokens for every edge and every path the key covers. (ii) A content owner using several CDNs must share the minting secret with each vendor, and each vendor can mint tokens that the others accept. (iii) There is no issuer accountability. Public-key tokens give edges only pk. We do not overstate the benefit. A compromised edge can already serve anything it caches without checking tokens, so asymmetric tokens remove only *lateral* minting: tokens accepted by other edges, other CDNs or other tenants. Table VIII in Section VII quantifies this blast radius for every baseline.

C. Adversary

The adversary $\mathcal{A}$ (i) controls the network between clients and edges and can replay observed tokens; (ii) may compromise *any subset* of edges and shields, learning their full state and injecting roots into their caches; (iii) is a legitimate client of any tenant and obtains arbitrarily many tokens for claims of its choice, while leaf salts remain issuer-chosen; and (iv) is quantum. It can run Shor's algorithm against any public key and generic quantum search against the hash function. $\mathcal{A}$ does not obtain the issuer's sk. Issuer compromise is handled by key rotation via kid. Theft and replay of bearer tokens are unchanged from the status quo and are mitigated by the same means: IP or ASN claims, short validity, and optionally a DPoP-style key-hash claim [20].

D. Goals and Requirements

G1 (unforgeability): no efficient (quantum) $\mathcal{A}$ makes an honest edge accept claims that I never issued or has revoked, even if all other edges are compromised. **G2 (edge non-minting):** edges hold no secret. **G3 (bounded validity):** an edge rejects a token whose expiry exceeds $T_{\text{start}}(e)+\Delta_e+V_{\max}$ (plus a clock-skew tolerance τ). **G4 (sibling privacy):** an authentication path reveals nothing about other clients' claims. **G5 (availability):** an edge that cannot obtain a root fails closed.

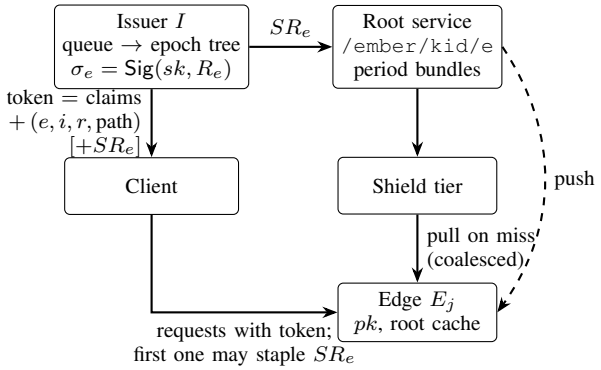


Fig. 1. EMBER architecture. The issuer signs one root per epoch. Edges obtain it by pull through the shield tier, by push, from a period bundle, or stapled by the client on its first request.

Operational requirements are: **R1**, tokens fit URL and cookie budgets with margin; **R2**, per-request edge cost close to HMAC; **R3**, no minting secret at edges; **R4**, bounded issuance latency; **R5**, revocation; **R6**, negligible dissemination bandwidth and rare root-miss waits; and **R7**, a clear security reduction.

IV. EMBER Design

Fig. 1 shows the architecture. The issuer collects token requests during an epoch e of length Δ_e, builds a Merkle tree over salted leaves, signs the root once and returns to each client a token containing its claims and authentication path. Edges verify a token by recomputing the root from the path and comparing it with a cached, signature-verified root.

A. Primitives and Notation

Let H be SHA-256 and $n \in \{16, 32\}$ the node length in bytes. We use the tweakable hash $\mathsf{Th}(T, x) = \text{trunc}_n(H(T \,\|\, x))$ with a fixed-length 26-byte tweak $T =$ "EMBERv1\0"$\|kid\|e\|type\|\ell\|j\|tenant$, where $type \in \{\text{LEAF}, \text{NODE}, \text{ROOT}, \text{PERIOD}\}$, ℓ is the tree level and j the node index. As in SLH-DSA's address-based domain separation [35], every hash call at every position of every epoch is a distinct function instance. The tenant identifier separates tenants that share a pooled tree. Leaf i of epoch e is $L_i = \mathsf{Th}(T_{\text{LEAF}}(e, i), r_i \,\|\, \text{enc}(c_i))$, where c_i are the canonical claims and $r_i \in \{0,1\}^{128}$ is a salt chosen by the issuer *after* it receives the claims. Inner nodes are $\mathsf{Th}(T_{\text{NODE}}(e, \ell, j), \text{left} \,\|\, \text{right})$. The tree has the RFC 9162 shape [34]: an unpaired node is promoted, so the path length is at most $d = \lceil \log_2 N_e \rceil$. The signed epoch root is

$$SR_e = (kid, e, T_{\text{start}}, \Delta_e, N_e, R_e, \sigma_e), \tag{1}$$

where

$$\sigma_e = \mathsf{Sig}\big(sk, \text{"EMBER-root"} \| kid \| e \| T_{\text{start}} \| \Delta_e \| N_e \| R_e\big), \tag{2}$$

Algorithm 1 Issuer: epoch closing and publication

procedure ISSUE(c)
  $r \leftarrow \$\{0,1\}^{128}$; append (c, r, future) to Q; **return** await future
end procedure
procedure CLOSEEPOCH ▷ timer Δ_e, or $|Q| = N_{\max}$
  $B \leftarrow Q$; $Q \leftarrow [\,]$; $e \leftarrow e+1$; $N \leftarrow |B|$
    ▷ rotate kid before e or i would wrap
  $L_i \leftarrow \mathsf{Th}(T_{\text{LEAF}}(e,i),\ B[i].r \,\|\, \text{enc}(B[i].c))$ for $i < N$
  $(R, \mathit{levels}) \leftarrow$ BUILDTREE(e, L) ▷ $N-1$ node hashes
  $\sigma \leftarrow \mathsf{Sig}(sk, \texttt{"EMBER-root"} \| kid \| e \| T_{\text{start}} \| \Delta \| N \| R)$
  PUBLISH(SR_e) at `/ember/`kid`/`e (immutable)
  if push mode: send SR_e to edges through the tier tree
  every m epochs: publish bundle PB_p, signed over $\text{MTH}(R_{e_1}, \ldots, R_{e_m})$
  for $i < N$ **do**
    $\pi_i \leftarrow$ PATH(levels, i)
    resolve $B[i].\mathit{future}$ with $(c_i, kid, e, i, r_i, \pi_i)$ $[+SR_e]$
  end for
end procedure

signed with the FIPS 204/205 context string `EMBER-root`. In the hybrid variant $\sigma_e = \sigma_{\text{Ed25519}} \| \sigma_{\text{ML-DSA-44}}$ over the same message, and both components must verify (an AND-combiner [30]).

B. Token Format

A token is a textual claim string followed by one base64url blob:

```
URLPrefix=...:Expires=...:Tenant=...
        [:IP=...]:E=blob
```

with $\mathit{blob} = \text{b64url}(ver_1 \| alg_1 \| kid_4 \| e_4 \| i_4 \| r_{16} \| \text{path}_{nd})$, i.e., $30 + nd$ raw bytes. With c_{enc} bytes of encoded claims, the token size is

$$S_{\text{tok}} = c_{\text{enc}} + 3 + \lceil 4(30 + nd)/3 \rceil. \tag{3}$$

Eq. (3) reproduces all 26 measured token sizes exactly (Section VII-A). The root signer does not affect token size.

C. Algorithms

Algorithm 1 gives issuance and Algorithm 2 edge verification. The issuer closes an epoch when its timer Δ_e fires, or earlier when the count trigger $N_{\max} = 2^{d_{\max}}$ is reached. The count trigger enforces the size budget: with budget B, $d_{\max} = \lfloor (3B/4 - 30 - c_{\text{enc}})/n \rfloor$.

Revocation. Validity is bounded by $V_{\max}$ (default 1 h). Beyond that, the issuer publishes signed revocation deltas as sets of (e, i) pairs, 8 B per revoked token, together with the period bundles. Edges check them with a hash-set lookup. A whole epoch can be revoked by withdrawing R_e, and the key can be rotated through kid.

Pooling. A CDN-operated issuer can place several low-rate tenants in one tree. This raises N_e and the chance that an edge already holds the root. Tenant is a claim and part of the leaf tweak, so a leaf cannot be replayed across tenants. Pooling is appropriate only when the tenant already trusts the CDN to mint its tokens.

Algorithm 2 Edge: per-request verification

procedure VERIFY($\mathit{tok}, \mathit{url}, \mathit{now}$)
  parse c and $(kid, e, i, r, \text{path})$; reject if malformed or kid unknown
  reject if $\mathit{now} > c.\mathit{Exp} + \tau$ or $c.\mathit{Exp} > T_{\text{start}}(e) + \Delta_e + V_{\max} + \tau$
  reject if $c.\mathit{URLPrefix}$ is not a prefix of url, or IP binding fails
  reject if $(e, i) \in \mathit{Revoked}$ or $i \geq N_e$
  $R \leftarrow \mathit{RootCache}[kid, e]$
  if $R = \perp$ **then**
    if stapled header `X-EMBER-Root` present **then** verify σ_e; insert
    else if e lies in a closed period p **then** fetch PB_p (coalesced); verify σ_p and MTH; insert all m roots
    else fetch SR_e via shield (coalesced, negative-cached); verify σ_e; insert
    end if
    on failure or timeout: **return** reject (fail closed)
  end if
  $x \leftarrow \mathsf{Th}(T_{\text{LEAF}}(e,i), r \| \text{enc}(c))$
  for each sibling s_ℓ in path **do** $x \leftarrow \mathsf{Th}(T_{\text{NODE}}(e, \ell, \cdot), \text{order}(x, s_\ell, i, \ell))$
  end for
  return $x \overset{\text{ct}}{=} R$ ▷ constant-time compare
end procedure

D. Security Argument

Definition 1 (Token unforgeability)**.** *In game* UF-TOK*, $\mathcal{A}$ receives pk, the stream of signed roots and period bundles, and the state of all edges. It adaptively queries* ISSUE(c) *and causes epochs to close. $\mathcal{A}$ wins if an honest edge's* VERIFY *accepts a token carrying (c^*, e^*, i^*) such that leaf i^* of epoch e^* was not issued for claims c^*, or (e^*, i^*) is revoked.*

Theorem 1. *Let $p \leq \sum_e (2N_e - 1)$ be the number of tree positions, counting period trees, at which the issuer evaluates* Th*. For every $\mathcal{A}$ that makes q_h queries to* Th*, there are adversaries $\mathcal{B}_1, \mathcal{B}_2$ with about the same running time such that*

$$\text{Adv}^{\text{uf-tok}}_{\text{EMBER}}(\mathcal{A}) \leq \text{Adv}^{\text{euf-cma}}_{\mathsf{Sig}}(\mathcal{B}_1) + \text{Adv}^{\text{sm-tcr}}_{\mathsf{Th},p}(\mathcal{B}_2), \tag{4}$$

where sm-tcr *is single-function, multi-target target-collision resistance with distinct tweaks [35]. Root and period signatures share sk but use distinct context strings, so a single EUF-CMA term covers both. For the hybrid root,* $\text{Adv}^{\text{euf-cma}}_{\mathsf{Sig}}$ *is at most the smaller of the two components' advantages [30].*

Proof sketch: Let R^* be the root that the edge used for (kid, e^*). *Case 1: the issuer never output R^* for (kid, e^*).* The edge verified σ on a message that the issuer never signed, either SR_{e^*} or a period message PB_p, which $\mathcal{B}_1$ outputs as a forgery. If PB_p is honest but lists R^*, the list hashes to the honest period MTH, which gives a period-tree collision at a fixed honest target; this is handled as in Case 2. *Case 2: $R^* = R_{e^*}$ is honest.* Since $i^* < N_{e^*}$ is enforced, honest leaf i^* exists with claims $c_{i^*} \neq c^*$. The honest and adversarial hash chains for position i^* start from different inputs and end at the same root, so at some position (e^*, ℓ, j) they have different inputs and equal outputs under the same tweak. The honest input at the lowest such position contains the salt r_{i^*}, which

the issuer chose after $\mathcal{A}$ fixed c_{i^*}, or contains a value derived from it. $\mathcal{A}$ therefore could not have prepared a colliding pair in advance; it must hit a fixed honest target, which $\mathcal{B}_2$ turns into an sm-tcr break. Because tweaks are distinct, the p targets give no multi-target speed-up [35]. Revoked (e^*, i^*) are rejected by line 5 of Algorithm 2. Cross-epoch, cross-tenant and cross-kid replays change a tweak component and fall under Case 2. Epoch and index counters must not wrap, so kid is rotated first. ∎

Plain collision resistance of Th also suffices when $n = 32$. That matches the proof structure of batch signatures [12], which we cite as the formal basis. We do not claim a new full proof. G2 holds structurally, and G4 holds because every sibling is a Th output over an input containing 128 fresh salt bits.

Truncation to $n = 16$. With 128-bit outputs, the generic sm-tcr bound is about $q_h/2^{128}$ classically and $O(q_h^2/2^{128})$ quantumly. This is the same generic level at which SLH-DSA-128s/f, which also uses 16-byte tweakable hashes, is placed in NIST category 1 [8]. Collision attacks, at about 2^{43} quantum cost for 128-bit outputs, would matter only if $\mathcal{A}$ controlled both inputs. Issuer salting prevents this, which is why the salt is mandatory for $n = 16$. Epoch-bounded validity (G3) additionally confines a useful forgery to $V_{\max}$ after the epoch. This is defense in depth and not part of the bound. We recommend $n = 16$ with ML-DSA-44 or SLH-DSA-128 roots, and $n = 32$ for category-3/5 deployments.

Other threats. Downgrade: once a tenant has switched to EMBER keys, edges must stop accepting classical tokens for that tenant, as with post-quantum origin authentication [27]. *Privacy:* N_e in a public root leaks per-tenant issuance volume. Padding to 2^d (at most 12% issuer slowdown in our Python implementation), pooling, or stapling-only distribution avoid this. The pair (e, i) is a per-token identifier, as a per-token signature already is. *Side channel:* root-fetch timing reveals whether a token of $(tenant, e)$ was recently used at an edge; push or stapling removes it. *DoS:* tokens naming fake epochs can trigger root fetches, and fake stapled roots trigger signature verifications. Section VII-H measures both.

Correctness was checked by 51 Python tests: round trips, bit-flip rejection in every claims byte and in salts, paths and indices, epoch, kid and tenant replays, skewed clocks, revocation, hybrid halves, stapled and negatively cached roots, and every baseline. A further 128 cross-implementation test vectors are verified identically by the Go port.

V. Root Dissemination and Epoch Control

An edge cannot verify a token until it holds the signed root of the token's epoch. Batch signatures avoid the problem by carrying the root signature in every message [12]. That would cost 2,420 B per token for ML-DSA-44, which is what EMBER exists to avoid. We therefore analyze four ways in which a fleet of E edges can obtain roots.

Pull on miss. An edge that lacks R_e fetches SR_e from the root service through the shield tier, with one fetch in flight per (kid, e) and negative caching. Roots are immutable objects, so the shield caches them. The first request of a session at a cold edge waits one edge→shield(→root service) round trip plus a signature verification.

Push. The issuer fans SR_e out to all edges. No request waits if the push arrives before the client's first request. The fleet bandwidth is $BW_{\text{push}} = E\,(|\sigma| + 40)/\Delta$, which grows as $1/\Delta$.

Period roll-up. Every m epochs (period $P = m\Delta$) the issuer signs one bundle with the m epoch roots ($mn + |\sigma|$ bytes). An edge that misses on an epoch of a closed period learns all m roots with one fetch and one verification, and tokens are unchanged.

Root stapling. The issuer returns SR_e with the token. The client sends it once, in header `X-EMBER-Root` (2.4 KB for ML-DSA-44), on its first request to an edge. The edge verifies σ_e and caches the root, so the first request costs one signature verification and no round trip. The idea is analogous to OCSP stapling and to MTL mode's full versus condensed signatures [14]. The stapled header is outside the URL and cookie budgets. A client re-routed to a new edge in mid-session may lack it, which in our model is what makes stapling misses nonzero.

A. Cost Model

Let λ be a tenant's token rate, w_v the traffic share of edge v, t_h one Th call, and t_s, t_v the root signing and verification times. Assume Poisson token arrivals and pull-only dissemination with one root per epoch. The expected number of root misses per token is then

$$M(\Delta) = \frac{1}{\lambda\Delta} \sum_{v=1}^{E} \left(1 - e^{-\lambda\Delta w_v}\right), \tag{5}$$

which tends to 1 when $\lambda\Delta \ll E$ (every token is first at its edge) and to $E/(\lambda\Delta)$ when $\lambda\Delta \gg E$. The user-visible added delay per token, in seconds, is

$$D(\Delta) = \underbrace{\Delta/2 + t_{\text{epoch}}}_{\text{issuance wait}} + M(\Delta)\, F_{\text{miss}}, \tag{6}$$

where $t_{\text{epoch}} = N t_{\text{leaf}} + (N-1)t_h + t_s + N t_{\text{ser}}$ and F_{miss} is the round trip to the root service plus t_v (pull), or only t_v (stapling). Per-request edge cost is $t_{\text{req}} = t_{\text{parse}} + (d+1)t_h + t_{\text{lookup}} + (\rho_v/Q_v)\, t_v$, where ρ_v/Q_v is the fraction of requests at edge v that need a new root. Issuer throughput per core tends to $1/(2t_h + t_{\text{ser}})$ as N grows, compared with $1/t_s$ for per-token signing.

B. Epoch Control: From Controller to Guideline

Eq. (6) suggests an interior optimum, because $\Delta/2$ grows while $M(\Delta)$ falls. We built an adaptive controller that, at each epoch close, estimates λ by an exponentially weighted moving average (EWMA) and picks Δ from a 25-point logarithmic grid in $[0.01, 60]$ s. The choice minimizes $\hat{D}(\Delta)$ subject to hard constraints: the token budget ($N \le N_{\max}$), signer capacity ($t_{\text{epoch}} \le k\Delta$), a p99 issuance-latency SLO and an optional push-bandwidth cap. It uses no weighted objectives. Before running the experiment we fixed a demotion rule: if the controller improved mean D over the best *fixed* Δ, chosen in

TABLE II
BASELINES (B1–B13) AND VARIANTS. ALL ARE IMPLEMENTED IN PYTHON; THOSE MARKED † ARE ALSO IMPLEMENTED IN THE GO PROTOTYPE.

ID	Scheme
B1†	HMAC-SHA256, key shared with all edges (EdgeAuth-like)
B2†, B3†, B4	Ed25519 (Media CDN), ECDSA P-256 and RSA-2048 (CloudFront)
B5†, B6†	ML-DSA-44 / ML-DSA-65 per token
B7, B8	SLH-DSA-SHA2-128s / 128f per token
B9†	Hybrid Ed25519 + ML-DSA-44 per token
B10	Multi-MAC: one 16-B HMAC tag per verifier domain, $k \in \{1, 3, 10\}$
B11†	Delegated short-lived Ed25519 key, certified per epoch by ML-DSA-44 (cf. delegated credentials [38])
B12†	Per-epoch HMAC key wrapped with ML-KEM-768 [39] to each edge
B13	MTL mode: condensed signature over a growing series [14]
B5v†	B5 plus an edge verified-token cache (LRU keyed by SHA-256 of the token)
B5c, B6c	B5/B6 split over two cookies

hindsight, by less than 10 ms or 10%, it would be reported as a guideline rather than a contribution. Section VII-F shows that the rule triggered. The controller always chose the smallest grid value, and a *work-conserving* policy did slightly better. This policy closes the epoch whenever the signer is idle and at least one request is pending.

VI. EVALUATION METHODOLOGY

A. Baselines

Table II lists the schemes. All of them authenticate the same canonical claim string, so only the authenticator differs. B1–B9 are the status quo and naive post-quantum schemes. B10–B13 are the strong alternatives that a fair comparison needs, and B5v, B5c and B6c are variants of B5 and B6. B14 (macaroons [19]) and B15 (DPoP- or TLS-bound tokens [20]) address attenuation and binding rather than unforgeability, so we discuss them only qualitatively. The EMBER configurations are O1 (ML-DSA-44 root, $n = 16$, the default), O2 ($n = 32$), O3 (SLH-DSA-128s root) and O4 (hybrid Ed25519 + ML-DSA-44 root).

B. Testbed and Implementations

All experiments ran on one KVM guest with 4 vCPUs (Intel Xeon at 2.1 GHz, with AVX-512 and SHA-NI) and 15 GB RAM, running Linux 6.18. Every timing names its implementation. The *primary* source for primitive timings is a native C benchmark against OpenSSL 3.5.4 using the EVP interface: 5 fresh processes pinned to one core, with warm-up. Each value is the median of the 5 run medians, with a bootstrap 95% confidence interval (CI) over the run medians, 2000 resamples. Signatures use 2000 iterations per run, 10 for SLH-DSA-128s signing, and path verification uses $2 \cdot 10^5$ iterations per run. The Go prototype uses Go 1.24.7 and cloudflare/circl 1.6.1. The Python reference uses pqcrypto 1.0.0 and serves only as a secondary source. Its ML-DSA-44 signing is 15.4× slower than native because the extension uses a portable Rust backend, so no claim about issuer speed rests on it.

Two native runs. The native suite ran twice, before and after a VM reboot, with identical scripts and pinning. Absolute values differ by 15–20%: ML-DSA-44 sign/verify took 501.8/122.3 µs in run 1 and 408.8/100.9 µs in run 2, and a $d = 16$ path took 2846.9 and 2341.2 ns. Ratios between schemes are stable. All simulations use run 1, the run available when they started. All primitive tables and figures use run 2. Each number in this paper is tagged with its run in the accompanying trace file.

C. Measurements

Token sizes are measured on generated tokens with a 60-character path and no IP claim (118-B canonical claims). **Wire bytes** per session are encoded with Go's `x/net/http2/hpack` (4096-B dynamic table, default indexing). A session has 1 manifest and $K = 150$ segment requests, plus about 570 B of other Chrome-like headers. We also report HTTP/1.1 and a 64 KiB table.

Go prototype. The prototype has an edge verifier (`net/http`, root cache with coalescing and a negative cache, optional verified-token LRU, V virtual edges with separate root caches), an issuer and root service with timer-Δ batching, and an open-loop load generator. Client, edge and origin run in three Linux network namespaces joined by two userspace link emulators, because `netem` is not available in this kernel. The emulated links have 10 ms and 37.5 ms one-way delay, 1 Gbit/s serialization and a 1500-B MTU. Measured keep-alive round-trip times are 20.98 ms (client→edge) and 76.10 ms (edge→origin). The edge runs on its own core (`GOMAXPROCS=1`). Three experiments use the prototype. *P1* measures closed-loop edge capacity over the edge namespace's loopback with 64 connections, 150 requests per token, 5 s warm-up and 20 s measurement, 5 runs. Its primary metric is *edge CPU time per request* (edge CPU utilization from `/proc` divided by req/s), which stays valid when the load generator, not the edge, limits throughput. *P2* measures open-loop latency over the emulated link at 50% and 80% of each scheme's P1 capacity. *P3* measures root dissemination: modes pull and pull_staple, $V \in \{10, 50\}$ virtual edges, $\Delta \in \{0.1, 1, 10\}$ s and $\lambda \in \{20, 200\}$ tokens/s, giving 24 configurations, each with a 60 s issuance window and 5 runs. CIs for prototype metrics are Student-t 95% intervals over the 5 runs.

Fleet simulator. The simulator is an epoch-level Monte Carlo engine with a numba kernel. It replays every first request and every re-route request against per-edge and per-shield root caches with coalescing, and draws CPU costs from the native measurements. Table III lists its parameters. All workload and network values are assumptions, and the sensitive ones are swept. We use 5 seeds (10 for the controller), Student-t CIs, and common random numbers across policies, so that comparisons are paired. An independent event-driven simulator agrees with it to within 0.56% on misses per token, 0.09% on D and 0.08% on issuance wait, across 45 metric rows. We report this only as an internal consistency check. The validation against *measured* behavior is the comparison with P3 in Section VII-G.

TABLE III
SIMULATION PARAMETERS (ASSUMPTIONS, NOT MEASUREMENTS). LN(m, s): LOGNORMAL WITH MEDIAN m AND LOG-STANDARD DEVIATION s.

Parameter	Default	Sweep
Edges E (shields $E/10$)	500	100, 500, 2000
Edge traffic share	Zipf(0.8)	0, 0.8, 1.2
Tenants, rate share	1000, Zipf(1.0)	Zipf 0.8–1.2
Aggregate rate $\bar{\lambda}$	2000 tokens/s	200, 2000, 20,000
Requests per session K	geometric, mean 150, 4 s apart	–
Edge re-route prob./segment	0.0005	0, 0.0005, 0.005
$V_{\max}$; period P	3600 s; 300 s	300–86400; 60–1800 s
RTT edge→shield; shield→root	LN(15, 0.5); LN(60, 0.5) ms	×0.5, ×2
Push propagation	LN(150, 0.5) ms	×0.5, ×2
Token receipt→first request	LN(120, 0.5) ms	–
Mean segment size	1.5 MB	0.5, 1.5, 4 MB
Node size; root signer	16 B; ML-DSA-44	32 B; ML-DSA-65, SLH-128s, hybrid
Controller	SLO 2 s, B = 1024 B, EWMA 0.2	–
Horizon	1 h (sweeps, 1200 s at $\bar{\lambda}$ = 20,000), 24 h (controller)	–

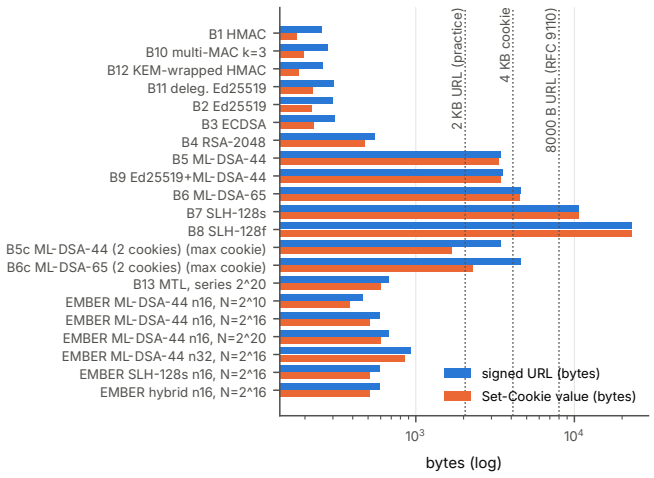


Fig. 2. Encoded token size per scheme (measured; 60-character path). Vertical lines: 2 KB URL practice, 4096-B cookie [10], 8000-B URL recommendation [17].

Hypotheses. We fixed nine hypotheses (H1–H9) and their thresholds before running the experiments. Table X in Section VIII reports every outcome, including those that failed.

VII. RESULTS

A. Token Size and Bytes on the Wire

Fig. 2 shows measured token sizes. Per-token post-quantum signatures do not fit: B5 (ML-DSA-44) is 3,350 B, B6 4,535 B, B7 10,598 B and B8 22,907 B, and the B9 hybrid is 3,435 B. Under a lognormal URL-path assumption (median 80 characters), no B5 or B6 signed URL fits in 2 KB, no B6 cookie fits in 4096 B, and B7/B8 fit no limit. Classical tokens are 166 B (B1), 209 B (B2) and 465 B (B4). The strong symmetric and delegated baselines are smaller than EMBER: B12 is 171 B, B11 214 B and B10 187 B for $k = 3$. EMBER O1 grows by about 21 B per tree level. It is 375 B at $N = 2^{10}$, 503 B at 2^{16}, 588 B at 2^{20} and 673 B at 2^{24}. With $n = 32$ (O2) it is 844 B at 2^{16}. EMBER is therefore 6.7× smaller than B5 at $N = 2^{16}$, but it stays under 512 B only up to $N = 2^{16}$, not up to 2^{20} as we had hypothesized. The MTL condensed signature (B13) has the same size at the same tree size (504 B for a 2^{16} series). The difference is that MTL's series grows with $\lambda V_{\max}$, whereas EMBER's tree grows only with $\lambda\Delta$. Eq. (3) matches all 26 measured EMBER sizes exactly (Fig. 3, top).

TABLE IV
UPSTREAM REQUEST-HEADER BYTES PER SESSION (1 MANIFEST + 150 SEGMENTS; MEASURED WITH GO HPACK, 4096-B TABLE UNLESS NOTED).

Scheme, placement	token B	h2	HTTP/1.1	h2, 64 KiB
No token	0	8,557	86,823	8,761
B1 HMAC, URL	127	26,706	107,057	24,918
B11 deleg. Ed25519, URL	175	33,277	114,305	30,825
B12 KEM-wrapped, URL	132	27,660	107,812	25,540
B5 ML-DSA-44, URL	3,311	457,831	587,841	410,359
B5 ML-DSA-44, cookie	3,311	458,568	589,955	11,667
B5c, 2 cookies	3,311	460,984	592,522	11,933
O1 N=2^{14}, URL	421	66,344	151,451	60,605
O1 N=2^{14}, URL + staple	421	69,298	154,749	63,149
O1 N=2^{14}, cookie	421	10,775	153,565	9,358
O1 N=2^{20}, URL	549	83,225	170,779	75,820

Fitting a budget is guaranteed by construction (the count trigger), so the more informative metric is bytes per session (Table IV). Token sizes in this experiment are smaller because of its different claim string. In URL form, an EMBER session uses 66,344 B under HTTP/2, which is 0.144× the split-cookie ML-DSA-44 baseline (B5c) and meets our hypothesis of at most 0.25×. Under HTTP/1.1 the ratio is 0.256× and narrowly misses it, because about 87 KB per session are token-independent headers. In cookie form the outcome depends on the HPACK table size. With the default 4096-B table, the 3.3 KB ML-DSA cookie is evicted by the changing `:path` entries and is re-sent on every request (3,036.9 B per request), and splitting it into two cookies does not help. With a 64 KiB table the cookie stays indexed, and the gap nearly vanishes: 11,667 B for B5 versus 9,358 B for EMBER. Stapling the root costs 2,954 B once per session under HTTP/2.

B. Edge Verification Cost

Fig. 4 and Table V give per-verification costs. Natively, ML-DSA-44 verification takes 100.9 µs [100.2, 120.4], close to Ed25519 (94.6 µs). A warm EMBER path at $d = 16$ takes 2.34 µs [2.33, 2.46]. That is 43× cheaper than ML-DSA-44 verification, but 7.9× the 0.30 µs of HMAC-SHA256 over the 200-B claims. Our hypothesis of at most 5× HMAC holds only for $d \leq 8$ (1.28 µs, 4.3×). Path cost grows by about 0.13 µs per level, from 0.75 µs at $d = 4$ to 3.41 µs at $d = 24$. The Go implementation gives a smaller ratio: a full EMBER token verification (parse plus path, $N = 2^{14}$) takes 3.074 µs, which is 3.6× Go HMAC verification (0.853 µs) and 10.7× cheaper than circl ML-DSA-44 verification (32.76 µs). circl's

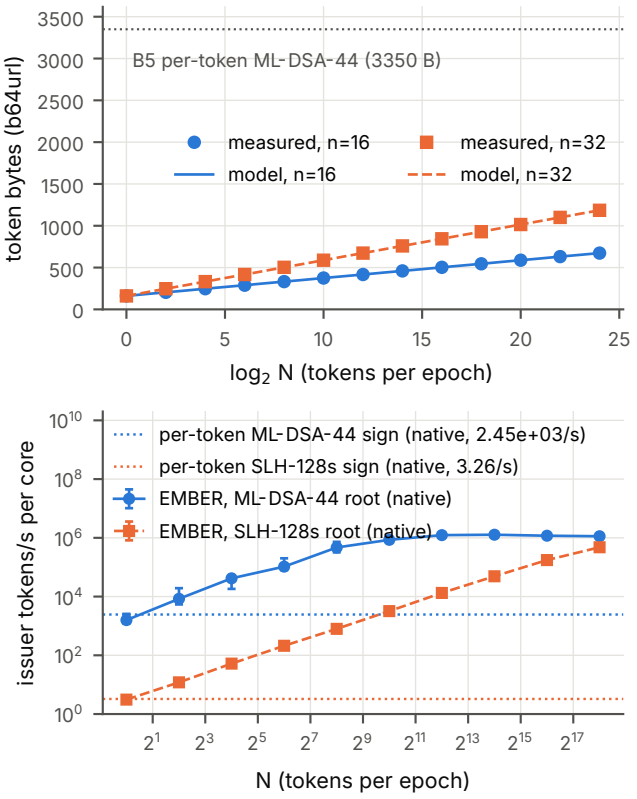


Fig. 3. Top: EMBER token size vs. $\log_2 N$ (markers measured, lines Eq. (3)). Bottom: native issuer throughput per core (OpenSSL 3.5.4, run 2), with per-token signing rates as horizontal lines.

TABLE V
NATIVE PRIMITIVE COSTS (OPENSSL 3.5.4, RUN 2; MEDIAN [95% CI], µS; SIZES IN BYTES).

Scheme	pk	sig	sign	verify
HMAC-SHA256	–	32	0.30 [0.30, 0.30]	0.30 [0.30, 0.30]
Ed25519	32	64	30.1 [30.1, 30.4]	94.6 [93.8, 96.2]
ECDSA P-256	65	71	23.0 [22.9, 31.1]	69.6 [69.5, 70.0]
RSA-2048	270	256	314.1 [309.4, 340.5]	20.7 [18.8, 24.2]
ML-DSA-44	1312	2420	408.8 [386.9, 452.4]	100.9 [100.2, 120.4]
ML-DSA-65	1952	3309	676.5 [673.5, 682.7]	154.8 [154.6, 165.8]
ML-DSA-87	2592	4627	808.7 [796.3, 871.5]	244.8 [243.5, 250.0]
SLH-DSA-128s	32	7856	306,464	288.1 [282.4, 320.0]
SLH-DSA-128f	32	17,088	14,436	873.2 [862.3, 883.6]
EMBER path d=16	–	286	–	2.34 [2.33, 2.46]

ML-DSA-44 verification is 3.6–4.0× faster than OpenSSL's, outside the 3× cross-implementation bound we set in advance. We attribute this to circl's optimized arithmetic and report it rather than hide it.

Whole-request edge cost (P1). Fig. 5 shows the cost of a complete HTTP request, including Go's request parsing and response, on a single saturated edge core. Edge utilization was at least 0.99 in every one of the 60 runs. A request carrying an HMAC token (B1) costs 20.54 µs [20.25, 20.82] of edge CPU. A warm EMBER request (O1) costs 22.91 µs [22.49, 23.33], 1.12× HMAC, and one core serves 43,355 req/s against 48,340 for HMAC (0.897×). Per-token ML-DSA-44 (B5) costs 66.24 µs [65.01, 67.46], 2.89× EMBER, at 15,040 req/s. Adding a verification cache (B5v) lowers this to 37.38 µs (26,620 req/s), still 1.63× EMBER, because every request must parse and hash a 3.3 KB token. Re-verifying the root every 1000 requests (O1a, 22.92 µs) or using a hybrid root (O4, 22.93 µs [22.39, 23.47]) is indistinguishable from O1. Per-request crypto and token size, not post-quantum verification as such, determine edge cost: in Go, per-token ML-DSA-44 (66.24 µs) is cheaper than Ed25519 (71.04 µs), B11 (70.36 µs) and ECDSA P-256 (98.04 µs).

C. Issuer Throughput

Per-token ML-DSA-44 signing limits a native issuer core to $1/408.8\,\mu s \approx 2{,}446$ tokens/s. EMBER with an ML-DSA-44 root issues 469,254 tokens/s per core [319,698, 729,692] at $N = 2^8$ and 1,275,990 [1,161,768, 1,282,009] at $N = 2^{14}$, 192× and 522× more (Fig. 3, bottom). At large N the cost is dominated by leaf hashing and base64url encoding. SLH-DSA-128s, which manages only $1/306.5\,\text{ms} \approx 3.26$ tokens/s when signing per token, becomes a usable root signer: 49,063 tokens/s at $N = 2^{14}$. Its 306 ms signature, however, lies on the issuance path. In simulation it raises D from 0.575 s to 0.927 s in pull mode at $\Delta = 1$ s. B11 is not comparable to EMBER on issuance, contrary to our hypothesis: native Ed25519 signing (30.1 µs) limits it to about 33,000 tokens/s, so EMBER issues at least 14× faster. B12 tokens are as cheap as HMAC to issue, but every epoch requires E key wraps, which take 37.6 ms at $E = 500$ in our Python implementation. The Go prototype confirms the trend independently: its issuer reaches about 432,000 tokens/s at $N = 1024$, 53× its per-token ML-DSA-44 signing.

D. Latency Under Load (P2)

Over the emulated 21 ms link at 50% of each scheme's capacity, no scheme's added median latency relative to HMAC is distinguishable from zero, except B2 (−1.25 ms) and B3 (−1.39 ms), which run at much lower absolute rates. At 80% load, O1's median is −3.24 ms [−6.82, 0.34] relative to HMAC, while per-token ML-DSA-44 (B5) adds 11.47 ms [5.09, 17.85]. B5v's median of 80.05 ms has a CI that spans zero because individual runs collapsed. B5 and B5v are the only schemes that push 3.3 KB URLs at 9.5k and 16.4k req/s, about 260 and 450 Mbit/s of request bytes through a userspace link emulator that sustained about 624 Mbit/s in a bulk test. We therefore attribute this tail to the link emulator, not to edge CPU, but we have not verified this. All runs completed with zero errors. Load levels were set from the earlier P1 capacities.

E. Dissemination and End-to-End Latency

Fig. 6 shows the trade-off that Eq. (6) predicts. Issuance wait follows $\Delta/2$. Root misses per token fall only slowly, because with $E = 500$ edges and 1,000 Zipf-distributed tenants, $\lambda_\tau \Delta \ll E$ for almost every tenant. Without re-routes the simulated misses equal Eq. (5) to 3–4 digits. With the default mobility assumption (about 7% of sessions switch edge once), the model under-predicts: at $\bar{\lambda} = 2000$ and $\Delta = 1$ s it gives 0.862 misses per token against 0.921 simulated.

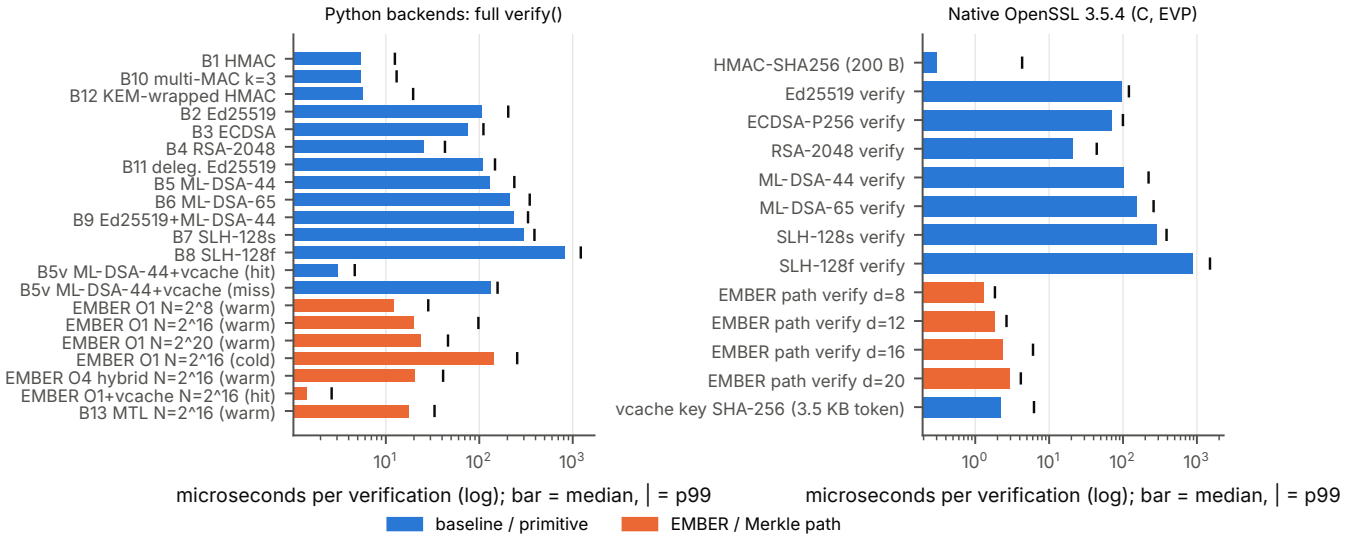


Fig. 4. Per-verification cost (bar: median; tick: p99). Left: Python reference, full `verify()` (secondary source). Right: native OpenSSL 3.5.4 primitives and EMBER path verification (primary source, run 2).

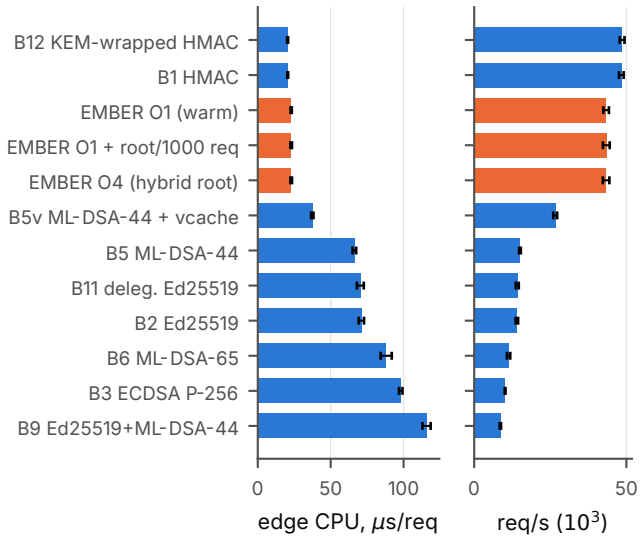


Fig. 5. Edge capacity of the Go prototype (P1, quiet rerun, one edge core, edge utilization ≥ 0.99 in all runs; mean and 95% t-CI over 5 runs). Left: edge CPU per HTTP request (primary metric). Right: requests/s.

TABLE VI
OPERATING POINTS (SIMULATED; $\bar{\lambda} = 2000$/s, $E = 500$, 5 SEEDS, CI HALF-WIDTHS $< 1\%$ OF THE MEAN).

Mode	Δ (s)	misses/token	D (ms)	req. waiting	root/served B
pull	0.01	1.069	96.2	0.71%	$2.4 \cdot 10^{-5}$
pull	1	0.921	575.4	–	–
pull_staple	0.01	0.073	11.7	0.049%	$1.6 \cdot 10^{-6}$
pull_staple	1	0.058	–	–	–
push	0.01	0.622	19.8	–	$5.0 \cdot 10^{-3}$
push_rollup	0.01	0.997	–	0.67%	$5.3 \cdot 10^{-5}$

Table VI and Fig. 7 compare the modes. **Stapling is the decisive mode.** It reduces misses per token from 1.069 to 0.073 at $\Delta = 0.01$ s, and the remaining misses come only from mid-session re-routes: 0 when no re-routes occur, 0.56 when the re-route probability is 0.005. It also cuts mean added delay D from 96.2 ms to 11.7 ms and the share of requests that wait for a root from 0.71% to 0.049%. At $E = 500$ it gives 387 misses per 10^6 requests, against 6,098 for pull. Root traffic is $1.6 \cdot 10^{-6}$ of served bytes, and fleet root bandwidth is 0.46–0.65 MB/s for $E = 100$–2000. **Push** at $\Delta = 0.01$ s needs 2.27 GB/s of root traffic. Even at $\Delta = 1$ s it needs 140 MB/s to 2.75 GB/s for $E = 100$–2000, and it still misses 0.622 per token at $\Delta = 0.01$ s, because the 150 ms push delay exceeds the 120 ms between token receipt and first request. **Roll-up** did not deliver the 5× miss reduction we hypothesized. It removes only 1.4–2.2% of misses (6,098→6,011 per 10^6 requests at $\Delta = 1$ s; 1.069→1.045 misses per token at $\Delta = 0.01$ s), because first requests arrive about 120 ms after issuance, before any period closes. Its value lies instead in bounding edge root state and in carrying revocation deltas (mean revocation propagation 150 s in pull modes). For comparison, B12's key-wrap push needs 57 MB/s to 1.13 GB/s and B11's certificate pulls 0.08–1.5 MB/s.

Fig. 8 shows the per-request added latency. With stapling, p99 is 0.0017 ms, 0.05% of requests exceed 1 ms and p99.9 is 0.12 ms. Plain pull has p99 0.123 ms and p99.9 112 ms, where root-miss round trips dominate. Per-token ML-DSA-44 adds 0.122 ms to *every* request, and B5v has p99 0.125 ms. Simulated edge CPU, including amortized root verification, is 1.2–1.7 µs per request for EMBER in pull mode.

Hybrid root. The hybrid root leaves warm per-request cost unchanged (20.27 vs. 19.98 µs in Python, +1.5%). In the Go prototype O4 and O1 differ by 0.09% per request, with overlapping CIs (Section VII-B). With per-tenant trees, however, each (edge, epoch) root serves about one session,

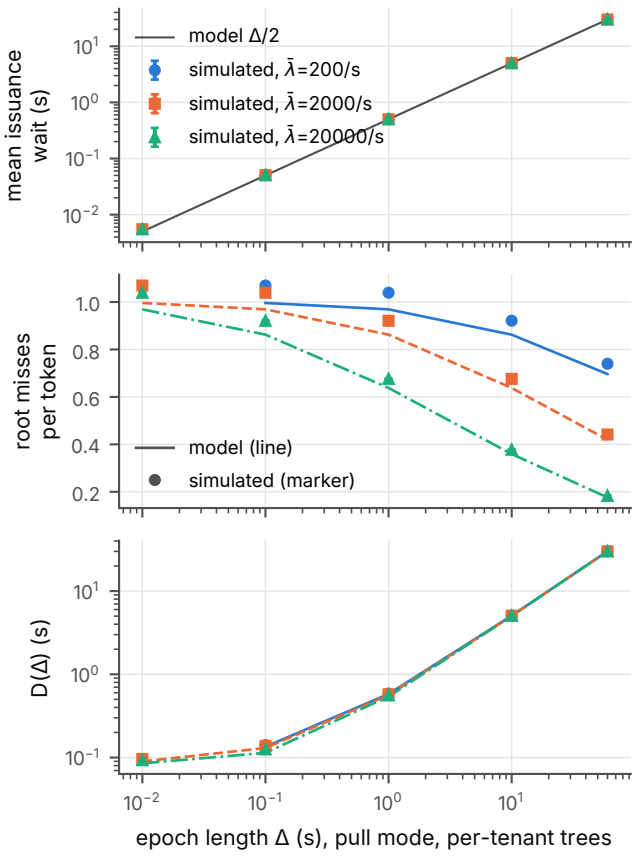


Fig. 6. Pull mode, per-tenant trees (simulated, 5 seeds; model lines from Eqs. (5)–(6)). Root misses barely fall with Δ until $\lambda\Delta$ approaches E, so $D(\Delta)$ is minimized at the smallest Δ.

TABLE VII
24-H DIURNAL WORKLOAD, PULL MODE (SIMULATED). D IN MS [95% t-CI]. PAIRED: ADAPTIVE MINUS POLICY (COMMON RANDOM NUMBERS).

$\bar{\lambda}$ (seeds)	adaptive D	best fixed D	paired diff.	WC D
200 (10)	96.61	96.61 (Δ=0.01)	−0.001 [−0.004, 0.002]	91.68
2000 (10)	96.08	96.08 (Δ=0.01)	+0.005 [0.001, 0.010]	91.68
20,000 (5)*	90.40	91.23 (Δ=0.01)	−0.83 [−0.91, −0.75]	91.45

*1-h peak window. Best fixed Δ chosen in hindsight per rate decile.

so the additional Ed25519 root verification raises *simulated* mean edge CPU per request by 41.6% (2.37 vs. 1.67 µs). The absolute cost remains at the microsecond level, but our hypothesis of a $< 1\%$ change is refuted.

F. Epoch Control

Table VII reports hypothesis H5, which failed. The adaptive controller chose the grid floor, $\Delta = 10$ ms, almost all day, and matched the best fixed Δ chosen in hindsight to within -0.83 to $+0.005$ ms (at most 0.9%). This is far below the pre-registered 10 ms / 10% threshold, so we demote the controller to a design guideline. A work-conserving close beat it by 4.93 ms and 4.41 ms at $\bar{\lambda} = 200$ and 2000. It was 1.0 ms worse at 20,000, where the adaptive policy was best. Fixed $\Delta \geq 3$ s violated the 2 s issuance SLO in every 5-min bin. The interior optimum predicted by Eq. (6) did not occur for any tenant at $\bar{\lambda} \leq 2000$, because misses do not fall until $\lambda_\tau \Delta \gg E$, while waiting grows linearly. With stapling, F_{miss} is nearly zero, and the optimum is trivially the smallest Δ the signer sustains.

G. Simulator Validation Against the Prototype

We ran the simulator with the prototype's P3 configuration and its measured RTT distribution, and compared it with the prototype (Fig. 9). For *misses per token* and *root bytes per token*, 24 of 24 configurations pass (relative error at most 15% or simulator inside the prototype CI). The maximum error is 4.4% for the simulator and 14.8% for the steady-state model, whose error at $\Delta = 10$ s comes from partly filled boundary epochs in the 60 s window. For example, at $V = 10$, $\Delta = 1$ s and $\lambda = 200$/s the prototype measures 0.05116 [0.05036, 0.05196] misses per token and the simulator predicts 0.05067. For *mean miss latency*, pull at $\Delta \leq 1$ s passes in 8 of 8 configurations (prototype 76.9–78.5 ms, simulator 75.9 ms). The other configurations fail. Pull at $\Delta = 10$ s measures 118–137 ms against 75.9 ms predicted. The four concurrent pipelines' timers are aligned, so each epoch close releases about 4,400 tokens whose first requests reach the one-core edge within about 200 ms. First-request p50 latency is 136.8 ms at $\Delta = 10$ s versus 23.6 ms at $\Delta = 1$ s. The simulator has no edge-CPU or link queueing. Stapled misses measure 0.074–0.086 ms against 0.026 ms predicted, because the simulator charges only signature verification, not decoding the 3.3 KB header. That absolute gap is about 0.05 ms. We therefore scope the simulator's latency claims to non-bursty edges. Its miss and byte predictions are validated.

H. Denial of Service

We flooded one simulated edge with 10,000 fake tokens/s. When the fake epochs are spread over all tenants, coalescing and negative caching do *not* help, because keys rarely repeat: 9270–9636 root fetches/s, 4630 signature verifications/s and 0.48 of one core. Restricted to the lowest-rate tenant, coalescing reduces fetches from 7680/s to 65/s. A stapled-root flood costs 10,000 verifications/s, 1.03 cores. A per-client limit of one staple per second (100 clients) reduces this to 100/s and 0.027 of a core. Per-client limits on fetches and staples are therefore required. Coalescing and negative caching protect only against single-tenant floods.

I. Trust and Cost Summary

Table VIII summarizes the comparison. The B5v hit cost (2.19 µs) is the SHA-256 cache key over a 3.5 KB token. EMBER is not the best scheme on every metric. B11 and B12 have smaller tokens, and HMAC-based schemes have cheaper verification. Among the schemes we compared, EMBER is the only one that is at once (i) post-quantum without a time-bounded classical assumption, (ii) free of minting secrets at edges and (iii) small enough for URLs and cookies. Its costs are a root-dissemination path and tokens about 3× larger than HMAC tokens.

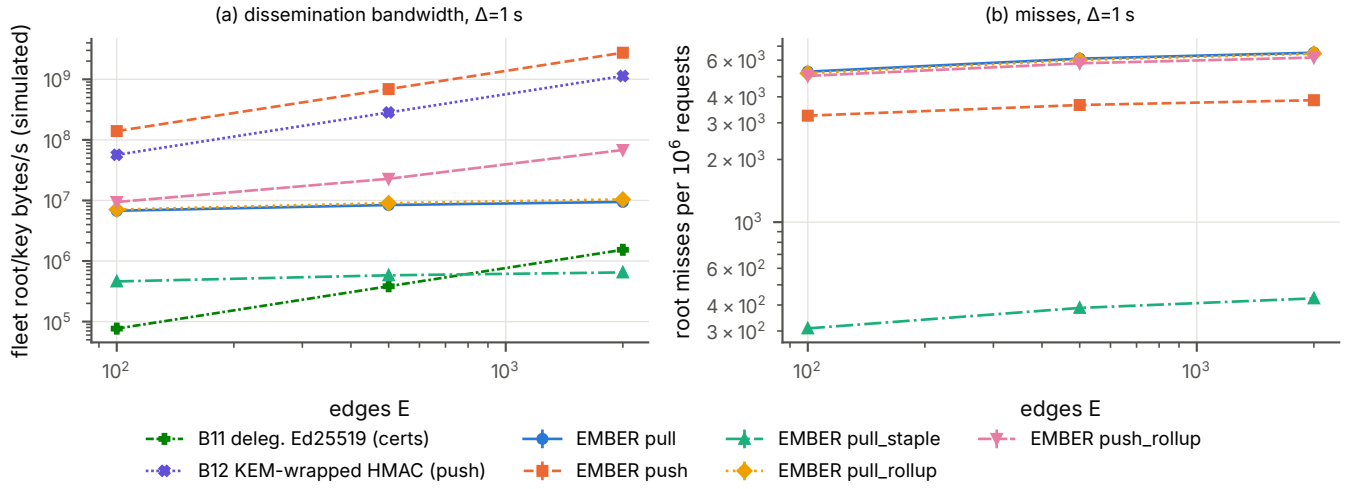


Fig. 7. Dissemination vs. fleet size ($\Delta = 1$ s, $\bar{\lambda} = 2000$/s; simulated). (a) Fleet root, key or certificate bytes/s. (b) Root misses per 10^6 requests.

TABLE VIII
TRUST AND COST COMPARISON. BYTES: MEASURED TOKEN, 60-CHARACTER PATH. VERIFY: NATIVE OPENSSL 3.5.4 MEDIAN PER REQUEST (EMBER: PATH, $d = 16$). DISSEMINATION: SIMULATED FLEET BYTES/S FOR $E = 500$, $\bar{\lambda} = 2000$/S, $\Delta = 1$ S.

Scheme	Secret at edge	One compromised edge mints for	PQ assumption	Bytes	Verify (µs)	Dissem. (kB/s)
B1 HMAC	yes (key lifetime)	all edges, all paths of the key	unconditional	166	0.30	0
B10 multi-MAC, k=3	yes	its own domain	unconditional	187	0.30	0
B12 KEM-wrapped HMAC	yes (epoch keys)	all edges, live epochs	unconditional (ML-KEM)	171	0.30	283,521
B2 Ed25519	no	none	none (Shor)	209	94.6	0
B11 delegated Ed25519	no	none	classical security within $V_{\max}$	214	94.6	383.2
B5 ML-DSA-44	no	none	unconditional	3,350	100.9	0
B5v + verify cache	no	none	unconditional	3,350	100.9 first, 2.19 hit	0
EMBER O1 (pull)	no	none	unconditional	503	2.34 (+root per epoch)	8,411
EMBER O1 (push)	no	none	unconditional	503	2.34	689,374

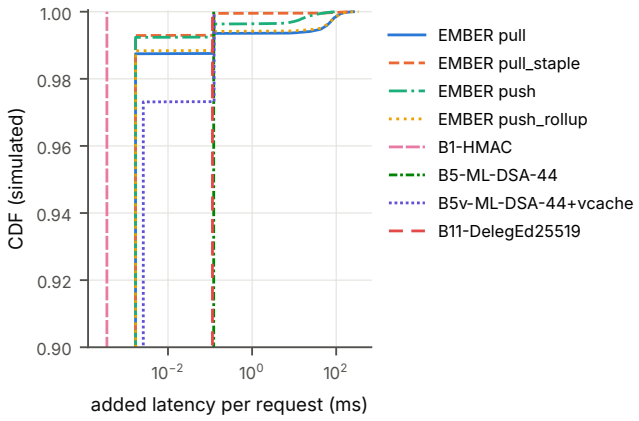


Fig. 8. Added latency per request (miss wait plus native verification CPU; simulated, $5 \times 20{,}000$ samples per curve). Upper tail shown.

When to choose something else. If one operator controls all edges and accepts a shared key, HMAC (B1) is post-quantum, cheapest and simplest. Per-domain multi-MAC (B10) contains a multi-CDN breach to one domain at HMAC cost. If an operator accepts the assumption that no CRQC breaks Ed25519 within the delegation lifetime, B11 gives 214-B tokens over the same dissemination path. B12 gives 171-B tokens, but it pushes a short-lived minting secret to every edge, at 283 MB/s of key traffic for $E = 500$ in our simulation. Per-token ML-DSA-44 with a verification cache (B5v) is competitive on CPU and, with a large HPACK table, on cookie bytes. Its cost lies in URL form, HTTP/1.1, the first request of each session, and 3.3 KB tokens that every request must parse and hash. EMBER is the option for operators who want edges to hold no secret and no classical-only assumption.

VIII. DISCUSSION

A. Deployment Guidance

Table IX lists the parameters we recommend on the basis of these measurements.

Integration. The signed root is a small, immutable, cacheable object, so it can be encoded as a COSE or JWS object with the ML-DSA algorithms of RFC 9964 [9]. Existing token-auth configuration (key ids, URL-prefix claims) carries over. The edge change is one hash loop and a root cache.

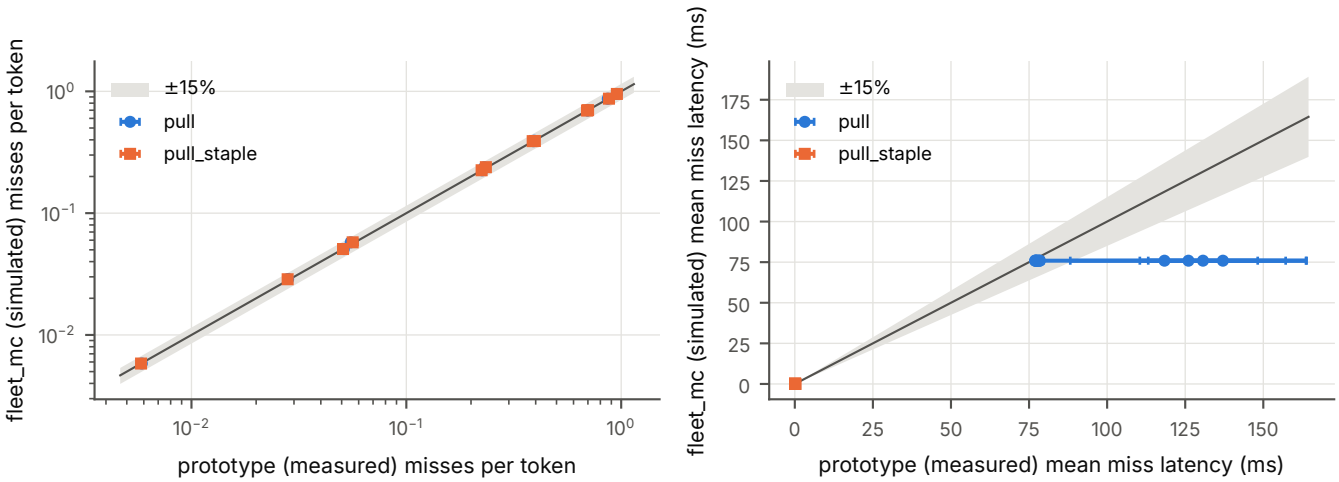


Fig. 9. Simulator (fleet Monte Carlo, 50 seeds, fed measured RTTs) vs. prototype (P3, 5 runs, 95% CI) on 24 configurations. Left: misses per token, 24/24 within 15% or inside the CI. Right: mean miss latency. Pull at $\Delta \leq 1$ s matches; pull at $\Delta = 10$ s and stapled misses do not.

TABLE IX
RECOMMENDED EMBER PARAMETERS.

Parameter	Recommendation and evidence
Node size n	16 B with ML-DSA-44 or SLH-DSA-128 roots (category 1); 32 B for category-3/5 profiles (+341 B at $N = 2^{16}$).
Root signer	ML-DSA-44. SLH-DSA-128s only if issuance can absorb a 306 ms signature per epoch. Hybrid root only if policy requires it (+41.6% simulated edge CPU with per-tenant trees).
Epoch close	Work-conserving, or a fixed Δ of about 10 ms, with count trigger $N_{\max} = 2^{16}$ (token $\leq$503 B for 118-B claims). Adaptive control brought no measurable gain.
Dissemination	Pull with stapling on the first request to each edge, coalesced pull as fallback. Avoid per-tenant push at small Δ (GB/s at $E \geq 500$).
Validity	$V_{\max}$ = 1 h; skew tolerance τ = 5 s; refresh tokens for longer sessions.
Period bundles	P = 300 s, used to carry revocation deltas and bound root state, not to reduce misses.
DoS	Per-client limits on fetches and staples; fetch only for configured kids and epochs within $[now - V_{\max} - P, now]$.
Transport	Prefer URL or header placement. With HTTP/2 cookies and a large HPACK table, the byte advantage over ML-DSA cookies is small.

TABLE X
PRE-REGISTERED HYPOTHESES AND OUTCOMES.

Hypothesis	Outcome	Evidence
H1 token <512 B to N=2^{20}; $\leq$ 0.25× B5c bytes	partly	503 B at 2^{16}, 588 B at 2^{20}; 0.144× (h2), 0.256× (HTTP/1.1)
H2 verify $\leq$ 5× HMAC, $\geq$ 10× cheaper than ML-DSA	partly	7.9× and 43× native; 3.6× and 10.7× Go; 1.12× HMAC per HTTP request
H3/H9 issuer $\geq$ 50× per-token signing; B11/B12 comparable	partly	192–522× native; B11 $\geq$14× slower
H4 root < 0.1% of bytes; roll-up $\geq$ 5× fewer misses	partly	$\leq 5.3 \cdot 10^{-5}$ except per-tenant push; roll-up 1.4–2.2%
H5 adaptive beats best fixed Δ	refuted	$\leq$0.83 ms; demoted
H6 < 1% requests wait; p99 CPU < 1 ms	supported	staple 0.049%
H7 hybrid root < 1% edge cost	mixed	+0.09% in P1 (warm); +41.6% simulated
H8 simulator within 15% of prototype	partly	misses 24/24; latency 8/24

B. Limitations

- *Single machine.* All prototype components share one 4-vCPU KVM guest. Network namespaces and a userspace link emulator stand in for netem, which is unavailable. No physical NIC is involved, and the emulator's throughput (about 624 Mbit/s in a bulk test) probably shaped the B5/B5v latency tails at 80% load. TLS is not included: connections are kept-alive HTTP/1.1, and the response body is 64 B, so P1 measures verification and request handling, not content delivery.
- *Timing variability.* The two native runs differ by 15–20% in absolute value across a VM reboot. Simulations use run 1 and primitive tables use run 2. Ratios are stable, but absolute values are implementation- and host-specific. circl verification is 3.6–4.0× faster than OpenSSL's.
- *Load-generator-bound capacity runs.* In the first P1 campaign, other experiments competed for the load generator's cores. In three of five runs the cheap schemes stopped saturating the edge (edge utilization 0.84–0.88), so their req/s values were lower bounds. We reran P1 on a quiet machine (load average 0.21 at the start, edge utilization $\geq$ 0.99 in all runs) and report only the rerun, with edge CPU per request as the primary metric. The rerun gives higher req/s than the first campaign for every scheme, so absolute capacity is sensitive to host conditions. P2 load levels were set from the first campaign's capacities, and P2 was not repeated.
- *Simulator scope.* The simulator has no edge-CPU or link queueing and does not charge for decoding the stapled header. It under-predicts pull-miss latency at $\Delta = 10$ s by

36–45% (118–137 ms measured vs. 75.9 ms) and staple-miss latency by about 0.05 ms. It is validated against the prototype only at small scale ($V \leq 50$), so results for $E = 2000$ are model-based extrapolations.

- *Workload assumptions.* We used no production CDN trace. Zipf exponents, RTTs, segment sizes, URL lengths and the re-route rate are assumptions, listed and swept in Table III. Low-rate tenants were not reported separately.
- *Security.* The security argument is a proof sketch that reduces to standard assumptions, not a machine-checked proof. Bearer-token replay is unchanged from the status quo. Pooling requires tenants to trust the CDN-held key. Epoch-bounded validity forces token refresh in long sessions.
- *Shortened prototype runs.* P3 used 60 s issuance windows instead of 5 min, and four concurrent pipelines. P2 used 15 s measurement windows.

IX. Conclusion

Per-token post-quantum signatures do not fit CDN access tokens, and HMAC, although post-quantum, leaves a minting key on every edge. EMBER applies a known primitive, Merkle batch signing, to this setting. It adds a token profile, dissemination of signed roots through the CDN, and stapling. With 2^{16} tokens per epoch, tokens are 503 B. Warm verification costs 2.34 µs natively. A complete HTTP request costs 1.12× HMAC in edge CPU in our Go prototype, against 2.89× for per-token ML-DSA-44. An issuer core produces 1.28 million tokens/s. Stapling reduces the share of requests that wait for a root to 0.049%. The measurements also bound the design. Verification is 7.9× HMAC rather than within 5×. Adaptive epoch control gave no benefit over a 10 ms or work-conserving epoch. Period roll-up hardly reduces misses. With large HPACK tables, ML-DSA cookies lose most of their byte penalty. Delegated Ed25519 and KEM-wrapped MAC keys remain smaller alternatives for operators who accept their trust assumptions. Future work includes measurements in a real multi-PoP deployment, a simulator with edge queueing, and binding tokens to TLS exporters.

Data Availability

All code (EMBER library, baselines, native C benchmarks, Go prototype, fleet simulator), raw measurements, aggregated CSV files and figure scripts are in the authors' repository. Every number in this paper is mapped to its source file and row in `NUMBERS_TRACE.md`. [Editable: insert public archive DOI or URL before submission.]

Declaration on the Use of AI Tools

[Editable: AI-based tools were used to assist with code, experiments and drafting. The author reviewed and takes responsibility for all content. Adapt to the venue's policy.]

Acknowledgment

[Editable.]

References

[1] Akamai Technologies, "EdgeAuth-Token-Java: Akamai edge authorization token," GitHub repository, hMAC-SHA256 token generation for cookie, query string and header tokens. Accessed 2026-11-05. [Online]. Available: https://github.com/akamai/edgeauth-token-java

[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[3] Amazon Web Services, "Specify signers that can create signed URLs and signed cookies (Amazon CloudFront developer guide)," Product documentation, key groups accept RSA-2048 or ECDSA-256 public keys. Accessed 2026-11-05. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-trusted-signers.html

[4] Google Cloud, "Media CDN: Generate signatures," Product documentation, ed25519-signed URLs, URL prefixes and cookies. Accessed 2026-11-05. [Online]. Available: https://cloud.google.com/media-cdn/docs/generate-signatures

[5] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[6] B. Westerbaan, "Cloudflare targets 2029 for full post-quantum security," Cloudflare Blog, Apr. 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-roadmap

[7] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," NIST, FIPS 204, Aug. 2024.

[8] ——, "Stateless hash-based digital signature standard," NIST, FIPS 205, Aug. 2024.

[9] M. Prorock and O. Steele, "ML-DSA for JSON object signing and encryption (JOSE) and CBOR object signing and encryption (COSE)," RFC 9964, May 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9964

[10] A. Barth, "HTTP state management mechanism," RFC 6265, Apr. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6265

[11] R. C. Merkle, "A certified digital signature," in *Advances in Cryptology – CRYPTO '89*, ser. LNCS, vol. 435. Springer, 1990, pp. 218–238.

[12] C. Aguilar-Melchor, M. R. Albrecht, T. Bailleux, N. Bindel, J. Howe, A. Hülsing, D. Joseph, and M. Manzano, "Batch signatures, revisited," in *Topics in Cryptology – CT-RSA 2024*, ser. LNCS, vol. 14643. Springer, 2024, pp. 163–186.

[13] A. Fregly, J. Harvey, B. S. Kaliski Jr., and S. Sheth, "Merkle tree ladder mode: Reducing the size impact of NIST PQC signature algorithms in practice," in *Topics in Cryptology – CT-RSA 2023*, ser. LNCS. Springer, 2023.

[14] J. Harvey *et al.*, "Merkle tree ladder (MTL) mode signatures," IETF Internet-Draft draft-harvey-cfrg-mtl-mode-09, work in progress, Mar. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-harvey-cfrg-mtl-mode-09

[15] D. Benjamin, D. O'Brien, B. Westerbaan, L. Valenta, and F. Valsorda, "Merkle tree certificates," IETF Internet-Draft draft-ietf-plants-merkle-tree-certs-02, work in progress, Mar. 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-ietf-plants-merkle-tree-certs-02.html

[16] D. Benjamin, "Batch signing for TLS," IETF Internet-Draft draft-ietf-tls-batch-signing-00, work in progress, Jan. 2020. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-tls-batch-signing

[17] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[18] R. Peon and H. Ruellan, "HPACK: Header compression for HTTP/2," RFC 7541, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7541

[19] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Network and Distributed System Security Symposium (NDSS)*, 2014.

[20] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, and D. Waite, "OAuth 2.0 demonstrating proof of possession (DPoP)," RFC 9449, Sep. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9449

[21] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, "Post-quantum privacy pass via post-quantum anonymous credentials," IACR Cryptology ePrint Archive, Paper 2023/414, 2023. [Online]. Available: https://eprint.iacr.org/2023/414

[22] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Network and Distributed System Security Symposium (NDSS)*, 2020.

[23] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography (PQCrypto 2020)*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.
[24] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, "The performance of post-quantum TLS 1.3," in *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT 2023)*. ACM, 2023, pp. 19–27.
[25] P. Kampanakis and W. Childs-Klein, "The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections," in *NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, 2024, iACR ePrint 2024/176. [Online]. Available: https://eprint.iacr.org/2024/176
[26] Akamai, "Future-proofing the Internet: Akamai achieves end-to-end PQC," Akamai Blog, Aug. 2026. [Online]. Available: https://www.akamai.com/blog/security/2026/aug/future-proofing-internet-akamai-achieves-end-to-end-pqc
[27] Cloudflare, "Post-quantum authentication to origins is now supported," Cloudflare Blog, Jun. 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-authentication-to-origins/
[28] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, 2020, iACR ePrint 2020/534.
[29] S. Celi, A. Faz-Hernández, N. Sullivan, G. Tamvada, L. Valenta, T. Wiggers, B. Westerbaan, and C. A. Wood, "Implementing and measuring KEMTLS," in *Progress in Cryptology – LATINCRYPT 2021*, ser. LNCS, vol. 12912. Springer, 2021, pp. 88–107.
[30] N. Bindel, U. Herath, M. McKague, and D. Stebila, "Transitioning to a quantum-resistant public key infrastructure," in *Post-Quantum Cryptography (PQCrypto 2017)*, ser. LNCS, vol. 10346. Springer, 2017, pp. 384–405.
[31] A. Fregly *et al.*, "SLH-DSA-MTL for DNSSEC," IETF Internet-Draft draft-fregly-dnsop-slh-dsa-mtl-dnssec-05, work in progress, Sep. 2025. [Online]. Available: https://www.ietf.org/archive/id/draft-fregly-dnsop-slh-dsa-mtl-dnssec-05.html
[32] L. Valenta, C. Patton, V. Gonçalves, and B. Westerbaan, "Keeping the Internet fast and secure: introducing Merkle tree certificates," Cloudflare Blog, Oct. 2025. [Online]. Available: https://blog.cloudflare.com/bootstrap-mtc/
[33] R. Burls, "Merkle tree agent certificates (MTAC)," IETF Internet-Draft draft-burls-mtac-01, work in progress, Oct. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-burls-mtac-01
[34] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," RFC 9162, Dec. 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9162
[35] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS$^+$ signature framework," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. ACM, 2019, pp. 2129–2146.
[36] P. Kampanakis and T. Lepoint, "Vision paper: Do we need to change some things? open questions posed by the upcoming post-quantum migration to existing standards and deployments," in *Security Standardisation Research (SSR 2023)*, ser. LNCS. Springer, 2023.
[37] N. Alnahawi, J. Müller, J. Oupický, and A. Wiesmaier, "A comprehensive survey on post-quantum TLS," *IACR Communications in Cryptology*, vol. 1, no. 2, 2024.
[38] R. Barnes, S. Iyengar, N. Sullivan, and E. Rescorla, "Delegated credentials for TLS and DTLS," RFC 9345, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9345
[39] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, FIPS 203, Aug. 2024.